Discussion Updates…Self-Signup starting 8 PM

- Our stable marriage algorithm **did not work** :-(
  - Much more constrained problem than previous semesters (fewer sections, fewer students)
- We will re-release **self-signup** directly in sections.cs61c.org **Today, Friday 1/30 8:00 pm PT**
  - We will **pre-assign Bridge** section (Nicolas, M 4-6pm, Soda 320)
  - We will **pre-assign Scholars** section (Cassie, Tu 1-2pm, Soda 405)
  - Everyone else will be **video section** (*we deeply apologize*)
  - Students in all sections can swap until Add/Drop deadline (**W 2/11**)
- We **strongly anticipate** that everyone who wants to do in-person discussion will (eventually) get into a section.
  - You can switch discussion times / modality until Add/Drop Deadline
  - If by Monday AM, you cannot attend any section that fits your schedule, follow instructions in **Week 3 Announcements**

CS61C

Great Ideas in Computer Architecture

(a.k.a. Machine Structures)

Assistant Teaching Professor
Lisa Yan

C Memory (Mis)Management

Section sign-ups start (again) today 8pm

Read K&R Chapter 5

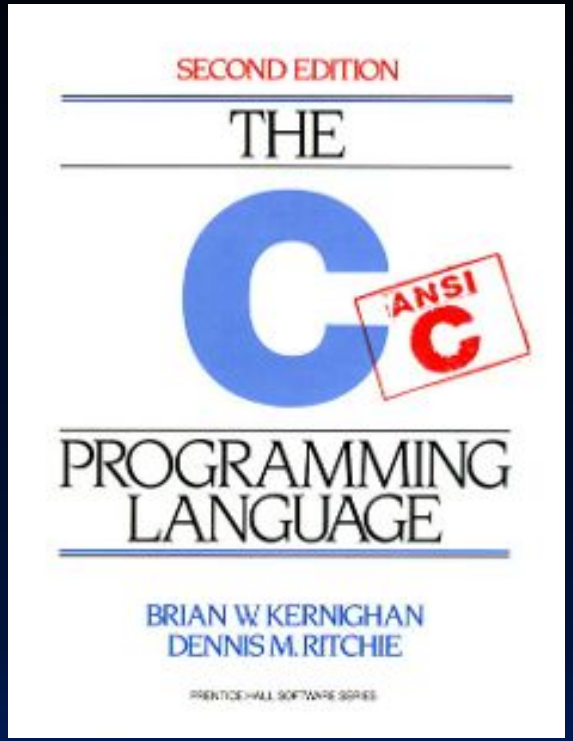
K&R:

> An array name is not a variable.

Meaning:

- Because pointers are variables, array names are not variables.
- However, arrays operate very similarly to pointers because

> Array names are synonymous with the location of the initial element.



(Two of) Three Array Properties

1. Array declarations set aside contiguous blocks in memory.

2. Array names are synonymous with the location of the first element in the array.

```
int arr[] = {795, 635};
```

arr

```
arr[1]; // *(arr+1)

uint32_t x = 4;
```

Array Bounds, in General

- Array bounds are **not checked** during element access.
  - Improper access may corrupt other parts of the program, including internal C data!

```
size_t n = 100;
int foo[n];
....
for(int i = 0; i <= n; ++i) {
    foo[i] = 0;
}
```

will write past end of array

WhatsApp Security Advisories

CVE-2019-11933

A heap buffer overflow bug in libpl_droidsonroids_gif before 1.2.19, as used in WhatsApp for Android before version 2.19.291 could allow remote attackers to execute arbitrary code or cause a denial of service.

click a link preview from a specially crafted text message.

Buffer overflow

(Three of) Three Array Properties

1. Array declarations set aside contiguous blocks in memory.

2. Array names are synonymous with the location of the first element in the array.

3. Arrays **decay** to pointers when used as function parameters or function arguments.

```
int arr[] = {795, 635};
```

arr

```
arr[1]; // *(arr+1)
```

(up next)

Agenda

Arrays and Functions

- Arrays and Functions
- C Strings

–

- Memory Locations
- The Stack
- The Heap

Arrays and Functions

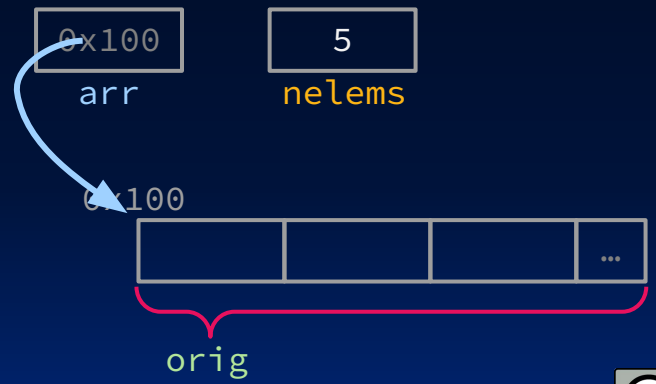
1. Formal array **parameters** are actually formal pointer parameters.
2. When passed as arguments to functions, array names "decay" to pointers.

1. same as `int *arr`

```
int bar(int arr[], size_t nelems     )
{
    … arr[0] …
}
int main(void)
{
    int orig[5], b[10];
    …
    bar(orig, 5);
    …
}
```

Bar doesn't know the size of the `orig` array! Need to specify size parameter!

2.

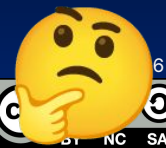


What gets printed?

`sizeof(arg)`: compile-time operator; gives size of `arg` in bytes

```
1  // for this exercise, assume hive architecture
2  // shorts are 16b, pointers are 64b
3  void mystery(short arr[], int len) {
4    printf("%d ", len);
5    printf("%d\n", sizeof(arr));
6  }
7
8  int main() {
9    short nums[] = {1, 2, 3, 99, 100};
10   printf("%d ", sizeof(nums));
11   mystery(nums, sizeof(nums)/sizeof(short));
12   return 0;
13 }
```

A. 10 5 10
B. 10 5 8
C. 80 5 80
D. 80 5 40
E. Other



What gets printed?

```
1  // for this exercise, assume hive architecture
2  // shorts are 16b, pointers are 64b
3  void mystery(short arr[], int len) {
4    printf("%d ", len);
5    printf("%d\n", sizeof(arr));
6  }
7
8  int main() {
9    short nums[] = {1, 2, 3, 99, 100};
10   printf("%d ", sizeof(nums));
11   mystery(nums, sizeof(nums)/sizeof(short));
12   return 0;
13 }
```

8: Array has decayed to a pointer

10: In array's declared scope, total array size.

5: In array's declared scope, # elements

A. 10 5 10
B. **10 5 8**
C. 80 5 80
D. 80 5 40
E. Other

Agenda

C Strings

- Arrays and Functions
- C Strings

–

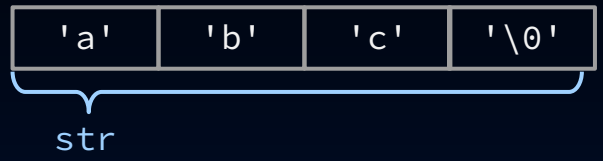
- Memory Locations
- The Stack
- The Heap

C Strings

A C string is just an array of characters, followed by **a null terminator**.

- Null terminator: the byte 0 (number) aka `'\0'` character)

```
char str[] = "abc";
```



The standard C library `string.h` assumes this convention.

- Example: `strlen` uses null-terminator under the hood and returns length without null-terminator.
- `strlen(str); // 3`

Unlike other arrays! By definition, C strings have an "end."

See notes for how string functions use null terminator

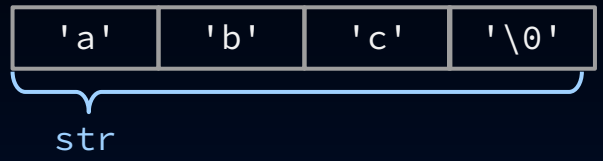
```
int arr[] = {795, 635};
```

C Strings vs. character arrays

A C string is just an array of characters, followed by **a null terminator**.

- Null terminator: the byte 0 (number) aka `'\0'` character)

```
char str[] = "abc";
```




```
char str2[] = {'a', 'b', 'c'};
strlen(str2); // undefined (where is
              // null terminator?
```

Comparison: a character array

(change of gears)

Agenda

Memory Locations

- Arrays and Functions
- C Strings

–

- Memory Locations
- The Stack
- The Heap

C Program Address Space [note: high mem on top]

- A program's **address space** contains 4 regions:
  - **Stack**: local variables inside functions, ***grows downward***
  - **Heap**: space requested via `malloc()`; resizes dynamically, ***grows upward***
  - **Data (aka Static)** segment: variables declared outside main, does not grow or shrink
  - **Text (aka Code)** segment: loaded when program starts, does not change
  - `0x00000000` chunk is unwriteable/unreadable so you that crash on `NULL` pointer access

For now, OS somehow prevents accesses between stack and heap.
(more later w/virtual memory)

Yan, SP26

C programming: Know <u>where</u> your data is stored

- Programming in C requires knowing where data is in memory.
  - Otherwise things don't work as expected.
  - By contrast, Java hides location of objects.
- Variable declared **inside function**:
  - **Local**, allocated on the **stack** and freed when function returns.
- Variable declared **outside function**:
  - **Global**, allocated in **data** segment.
- "**Automatic memory management**" for both:
  - You don't need to worry about deallocating when you are no longer using them.
  - ...Meaning, C will reuse stack memory for you...

Pitfalls of not managing memory

- The runtime **does not** check for the programmer's failure to manage memory.
  - Memory is so performance-critical that early on, C designers decided there isn't time to run background management
  - Usual result: you corrupt heap allocator's internal structure, and you find out much later in a totally unrelated part of your code!

Agenda

The Stack

- Arrays and Functions
- C Strings

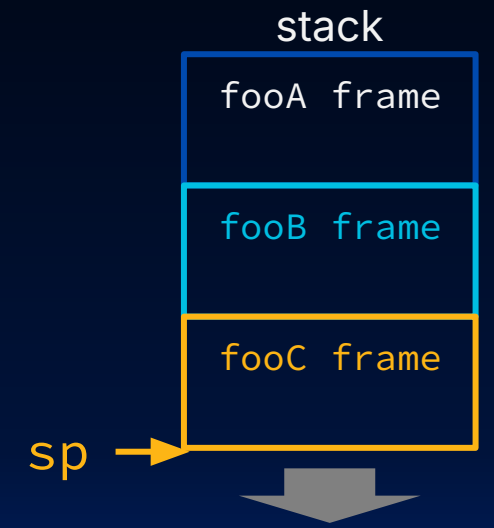
–

- Memory Locations
- The Stack
- The Heap

The Stack

- Every time a function is called, a new **stack frame** is allocated on the stack.
  - Contiguous block of memory
  - Local variables*
  - Return "instruction" address—who called me?*
  - Function arguments*
- When the function ends, the stack frame is "tossed off the stack."
  - "Frees" memory for future stack frames.
- The **stack pointer** tells us the "top of the stack," i.e., start of current stack frame.

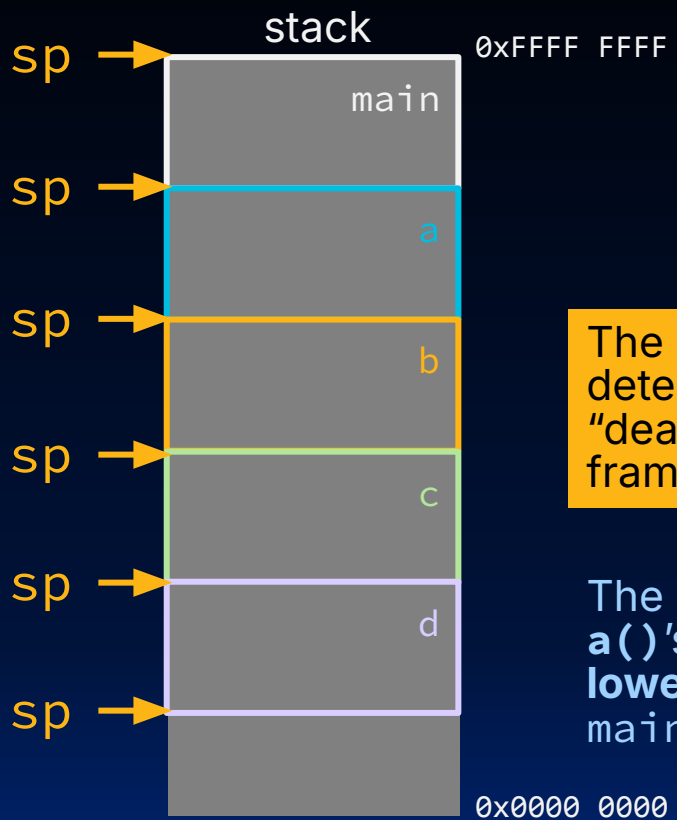
```
fooA() { fooB(); }
fooB() { fooC(); }
fooC() { … }
```



*more later when we cover details of a RISC-V architecture.

The Stack is Last In, First Out (LIFO)

```
int main ()
{ a(0);  …
}
void a (int m)
{ b(1);
}
void b (int n)
{ c(2);
}
void c (int o)
{ d(3);
}
void d (int p)
{
}
```



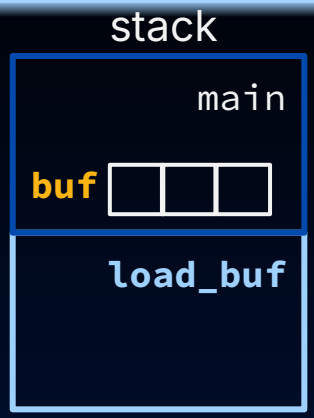
The **stack pointer** (sp) is what determines "allocating" and "deallocating/freeing" stack frames!

The stack **grows downward; a()'s local variables have lower byte addresses** than main()'s, and so on.

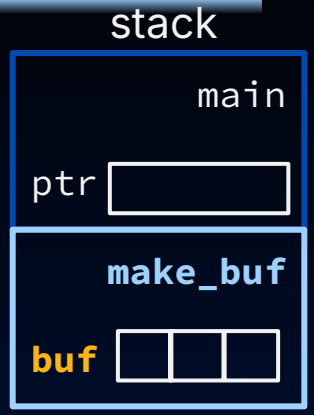
[animate this slide]

Passing pointers into the stack

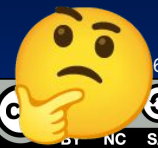
```
void load_buf(char *ptr,
              ...) {
  ...
};

int main() {
  ...
  char buf[...];
  load_buf(buf, BUFLEN);
  ...
}
```




```
char *make_buf() {
    char buf[50];
    return buf;
}
void foo(...) {...}
int main(){
   char *ptr = \
      make_buf();
   foo(ptr);
   ...
}
```

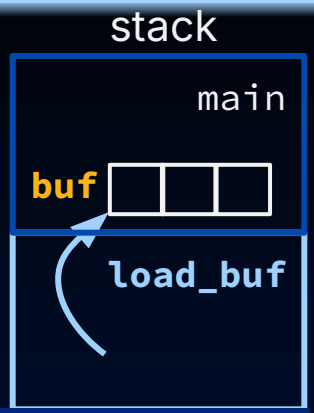


Why is the left code okay? Why is the right code dangerous?



Passing pointers into the stack

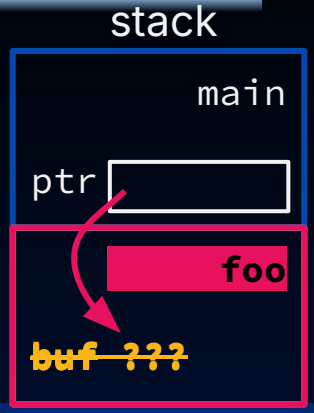
```
void load_buf(char *ptr,
              ...) {
  ...
};

int main() {
  ...
  char buf[...];
  load_buf(buf, BUFLEN);
  ...
}
```



buf persistent through load_buf's execution

It is fine to pass pointers to something in the stack as function arguments.

```
char *make_buf() {
    char buf[50];
    return buf;
}
void foo(...) {...}
int main(){
   char *ptr = \
      make_buf();
   foo(ptr);
   ...
}
```



ptr points to overwritten memory

It is catastrophic to return a pointer to something in the stack!

- Stack memory **is overwritten** when other functions are called!
- **Dangling reference**. Your data is no longer guaranteed to exist!

Yan, SP26

Agenda

[FOR NEXT TIME]

The Heap

- Arrays and Functions
- C Strings

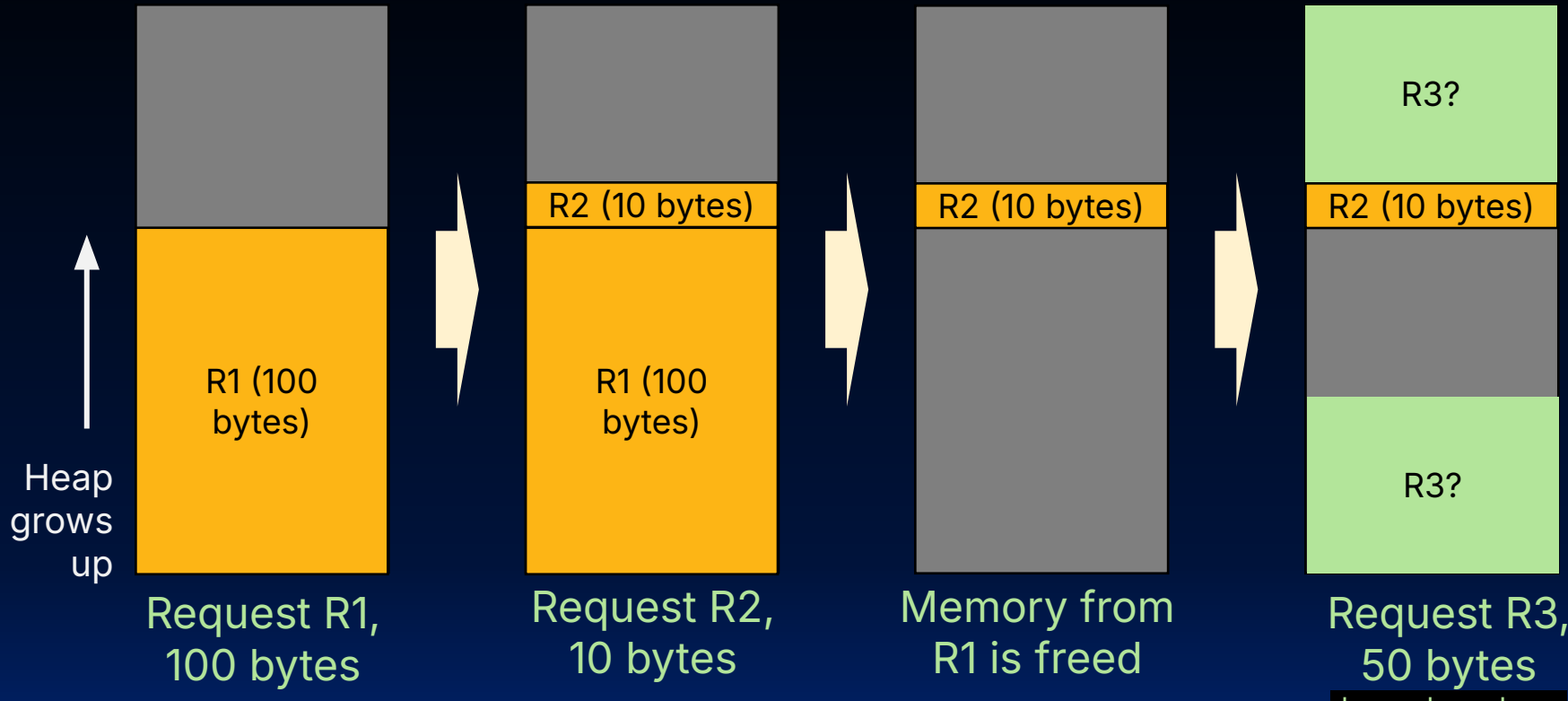
–

- Memory Locations
- The Stack
- The Heap

What is the Heap?

- The heap is **dynamic memory**: can be allocated, resized, and freed during program runtime.
  - Useful for persistent memory across function calls.
  - Similar to Java `new` command, which allocates memory…
  - …but key differences lead to biggest source of C memory leaks/bugs.
- Huge pool of memory (usually >> stack) **not** allocated in contiguous order.

Heap Management Example



stdlib.h heap

- C heap standard library functions: specify # bytes of memory explicitly to allocate/reallocate.
  - `malloc()`: Allocates raw, uninitialized memory from heap
  - `free()`: Frees memory on heap
  - `realloc()`: Resizes previously allocated heap blocks to new size

Unlike the stack, heap memory gets reused *only* when programmer explicitly cleans up.

Advice: Read the manual pages on hive

```
MALLOC(3)                  Linux Programmer's Manual                  MALLOC(3)

NAME
       malloc, free, calloc, realloc, reallocarray - allocate and free dynamic
       memory

SYNOPSIS
       #include <stdlib.h>

       void *malloc(size_t size);
       void free(void *ptr);
       void *calloc(size_t nmemb, size_t size);
       void *realloc(void *ptr, size_t size);
       void *reallocarray(void *ptr, size_t nmemb, size_t size);
```

Pro Tip: run **`man malloc!`** The Linux manual page describes the heap API (and undefined behavior).

```
int *ptr = malloc(20*sizeof(int));
```

Let's briefly discuss `stdlib.h` heap API

void *malloc(size_t n)

- Allocates a block of uninitialized memory:
  - **`size_t n`**: unsigned integer type big enough to "count" memory bytes.
  - Returns **`void *`** pointer to block of memory on heap.
  - A return of **`NULL`** indicates no more memory (always check for it!!!)
- To allocate, say, a struct:

```
typedef struct { ... } TreeNode;
TreeNode *tp = malloc(sizeof(TreeNode));
```

`sizeof(type)`: size in bytes.

- To allocate an array of 20 `int`s:

```
int *ptr = malloc(20*sizeof(int));
```

Assuming size of objects can lead to misleading, unportable code. Use `sizeof()`!

void free(void *ptr)

- Dynamically frees heap memory
  - `ptr` is a pointer containing an address originally returned by `malloc()/realloc()`.

```
int *ptr = (int *) malloc (sizeof(int)*20);
...
free(ptr); // implicit typecast to (void *)
```

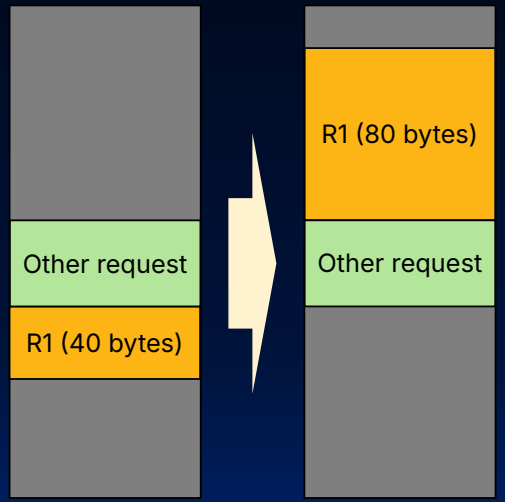
- When you free memory, be sure to pass the **original address** returned from `malloc()`. Otherwise, crash (or worse!)

void *realloc(void *ptr, size_t size)

- Resize a previously allocated block at `ptr` to a new size.
  - Returns new address of the memory block.
    - In doing so, it may need to copy all data to a new location.

- Remember: Always check if returned pointer is `NULL`—which would mean you've run out of memory!

```
int *ip; ip = (int *) malloc(10*sizeof(int));
… … …                    /* check for NULL */
ip = (int *) realloc(ip, 20*sizeof(int));
/* contents of first 10 elements retained */
… … …                    /* check for NULL */
```



Agenda

When (Heap) Memory Goes Bad

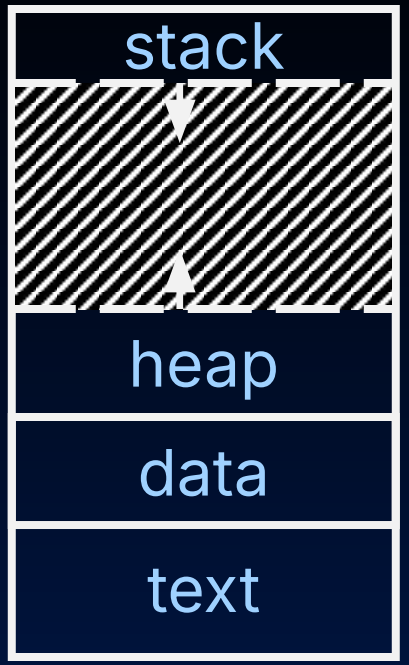
- Arrays and Functions
- C Strings

–

- Memory Locations
- The Stack
- The Heap

Working with Memory

- ✅ Code, Static storage are easy:
  - They never grow or shrink.
- ✅ Stack space is also "easy":
  - Stack frames are created/destroyed in LIFO order.
  - Just avoid **dangling references**: pointers to deallocated variables (e.g., from old stack frames).
- ⚠ Working with the heap is **tricky**.
  - Memory can be allocated / deallocated at any time!
  - **Memory leak**: You forget to deallocate memory } Will eventually run out of memory
  - "**Use after free**": (self-explanatory)
  - "**Double free**": Call free 2x on same memory } Possible crash or exploitable vulnerability



Faulty Heap Management

How many memory management errors are in this code?

```
void free_mem_x() {
  int fnh[3];
  ...
  free(fnh);
}

void free_mem_y() {
  int *fum = malloc(4*sizeof(int));
  free(fum+1);
  ...
  free(fum);
  ...
  free(fum);
  }
```

A. 1
B. 2
C. 3
D. 0
E. Other

Pull up `man malloc` on hive (hit q to exit)

L05: How many memory management errors are in this code?

```
void free_mem_x() {
  int fnh[3];
  ...
  free(fnh);
}

void free_mem_y() {
  int *fum = malloc(4*sizeof(int));
  free(fum+1);
  ...
  free(fum);
  ...
  free(fum);
  }
```

A. 1 — 0

B. 2 — 0

C. 3 — 0

D. 0 — 0

E. Other — 0

Faulty Heap Management

How many memory management errors are in this code?

```
void free_mem_x() {
  int fnh[3];
  ...
  free(fnh);
}

void free_mem_y() {
  int *fum = malloc(4*sizeof(int));
  free(fum+1);
  ...
  free(fum);
  ...
  free(fum);
  }
```

`free()` on stack-allocated memory

`free()` on memory that isn't the pointer from `malloc`

Double `free()`

A. 1
B. 2
C. 3
D. 0
E. Other

Valgrind to the rescue!!!

- Valgrind slows down your program by an order of magnitude, but…
- Valgrind adds a tons of checks designed to catch most (but not all) memory errors
  - Memory leaks
  - Misuse of free
  - Writing over the end of arrays
- Tools like Valgrind are absolutely essential for debugging C code.

Check out Lab 02!

And in Conclusion…

- C has 3 pools of memory
  - Static storage: global variable storage, basically permanent, entire program run
  - The Stack: local variable storage, parameters, return address
  - The Heap (dynamic storage): malloc() grabs space from here, free() returns it.
- Heap data is biggest source of bugs in C code
- `malloc()` handles free heap space with freelist. Take CS162 for more!